PROJET FIL ROUGE

API REST de gestion de bibliothèque

Cours de Bachelor 3 — Développement Back-end

Spring Boot 4.1.1 · Java 25

Durée	Jours	Prérequis
1,5 jour (9 h)	**Jour 4 après-midi et Jour 5**	**Parties 1 à 4**

3 ressources · 20 endpoints · 33 questions · recette de 46 requêtes

1. Présentation du projet

1.1 Contexte

Une bibliothèque municipale veut remplacer ses fiches papier par une application. L'équipe frontend (web et mobile) a besoin d'une **API REST** pour gérer :

- les **auteurs** ;
- les **livres** du catalogue ;
- les **emprunts** de livres par les lecteurs.

Votre binôme est chargé de concevoir, développer, documenter et tester cette API.

1.2 Ce que vous allez produire

Livrable	Contenu	Questions
Dossier de conception	Le fichier `CONCEPTION.pdf` : vos réponses aux questions de conception	Q1 à Q9
API Spring Boot	Le projet Maven `bibliotheque-api` : 3 ressources, **20 endpoints**	Q10 à Q29
Documentation	La documentation Swagger UI générée à partir de votre code	Q30 et Q31
Recette	Une collection Postman de **46 requêtes** avec le code HTTP attendu, **ou** le fichier `RECETTE.pdf` contenant les captures d'écran du Collection Runner	Q32
Client Java	Un programme `ClientBibliotheque` qui appelle votre API avec `HttpClient`	Q33

1.3 Ce que le projet mobilise

Partie du cours	Notions utilisées dans le projet	Questions
Partie 1 — API distante	Requête et réponse HTTP, JSON, méthodes et codes de statut, curl et Postman, `HttpClient` en Java	Q3, Q5, Q9, Q32, Q33
Partie 2 — Fonctionnement d'une API REST	Ressources, nommage des URLs, CRUD et méthodes HTTP, choix des codes de statut, sans état (*stateless*), versionnement `/api/v1/`	Q1 à Q4, Q7, Q20, Q24, Q29
Partie 3 — Spring Boot	Projet Spring Initializr, architecture Controller → Service → Repository, entités JPA, Spring Data, H2, `data.sql`, `ResponseEntity`, `@Valid`	Q10 à Q29
Partie 4 — Bonnes pratiques	DTOs de requête et de réponse, classe de conversion (mapper), pagination et tri avec `PagedModel`, documentation OpenAPI / Swagger	Q6, Q8, Q17, Q18, Q21, Q22, Q26, Q27, Q30, Q31

1.4 Ce que le projet ne vous demande pas

Pour que vous sachiez exactement où concentrer vos efforts, voici ce qui est **hors du périmètre** du projet. N'ajoutez aucun de ces éléments.

Hors périmètre	Ce que vous faites à la place
Expliquer ou configurer le conteneur IoC	Le début de chaque service et de chaque contrôleur (attributs + constructeur) est **fourni** : recopiez-le tel quel. Spring remplit automatiquement les paramètres du constructeur.
Filtres, intercepteurs, parcours d'une requête	Rien : aucun filtre ni intercepteur dans ce projet.
Tests unitaires (Mockito)	Rien : vous testez l'API avec Postman (Q32) et avec votre client Java (Q33).
Exceptions personnalisées, `@RestControllerAdvice`, format d'erreur JSON	Une erreur est signalée **uniquement par son code HTTP**, avec une réponse sans corps (§4.1).
Relations JPA (`@ManyToOne`, `@OneToMany`)	Un livre désigne son auteur par un simple nombre : `auteurId`. Un emprunt désigne son livre par `livreId`.

1.5 Organisation : trois sprints sur 1,5 jour

Le projet se fait **en binôme**. Chaque sprint se termine par une courte démonstration au formateur.

Sprint	Créneau	Contenu	Démonstration de fin de sprint
Sprint 1 — Conception et auteurs	Jour 4, 13 h 30 – 17 h 00	Parties A, B, C et D (Q1 à Q20)	`CONCEPTION.pdf` rempli ; requêtes R01 à R10 conformes
Sprint 2 — Livres et emprunts	Jour 5, 09 h 30 – 12 h 00	Parties E et F (Q21 à Q29)	Requêtes R01 à R46 conformes
Sprint 3 — Documentation et clients	Jour 5, 13 h 30 – 15 h 15	Parties G, H et I (Q30 à Q33)	Swagger UI complet ; client Java : 10/10 étapes conformes
Soutenances	Jour 5, 15 h 30 – 16 h 45	Partie J	5 min de démonstration + 2 min de questions par binôme

> **Règle de fin de sprint :** le projet doit **démarrer** et **répondre** à la fin de chaque sprint. Un sprint inachevé mais qui démarre vaut mieux qu'un sprint complet qui ne compile pas.

1.6 Ce que vous rendez

Une archive nommée `API-REST-<nom1>-<nom2>-<école>-<mois>-<année>.zip` (exemple : `API-REST-dupont-martin-coda-orleans-11-2026.zip`) qui contient le dossier du projet **sans le dossier `target/`** :

```
bibliotheque-api/
├── CONCEPTION.pdf                          ← Q1 à Q9
├── RECETTE.pdf                             ← Q32, choix 2 (si vous ne rendez pas la collection)
├── pom.xml
├── client/
│   └── ClientBibliotheque.java             ← Q33 (hors de src/, il ne fait pas partie de l'API)
├── postman/
│   └── Bibliotheque-API.postman_collection.json ← Q32, choix 1
└── src/
    ├── main/java/com/formation/bibliotheque/
    │   ├── BibliothequeApplication.java
    │   ├── config/        OpenApiConfig.java
    │   ├── controller/    AuteurController, LivreController, EmpruntController
    │   ├── dto/           AuteurRequest, AuteurResponse, LivreRequest, LivreResponse,
    │   │                  EmpruntRequest, EmpruntResponse
    │   ├── mapper/        AuteurMapper, LivreMapper, EmpruntMapper
    │   ├── model/         Auteur, Livre, Emprunt
    │   ├── repository/    AuteurRepository, LivreRepository, EmpruntRepository
    │   └── service/       AuteurService, LivreService, EmpruntService
    └── main/resources/
        ├── application.properties
        └── data.sql
```

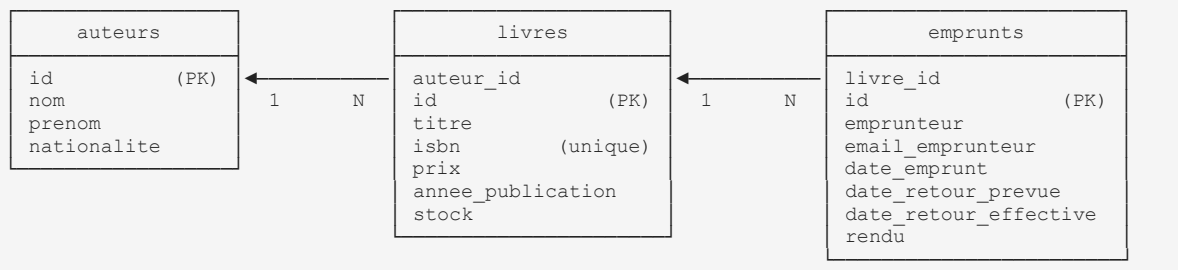
2. Règles métier

Ces règles sont numérotées : les questions y font référence (« appliquez RM5 »).

N°	Règle
RM1	Un livre a **un seul auteur**, désigné par son identifiant `auteurId`. Un auteur peut avoir plusieurs livres.
RM2	Deux livres ne peuvent pas avoir le même **ISBN**.
RM3	Un auteur qui a **encore au moins un livre** ne peut pas être supprimé.
RM4	Un livre est **disponible** si son stock est **strictement supérieur à 0**. Seul un livre disponible peut être emprunté.
RM5	Quand un livre est emprunté : son stock **diminue de 1** ; `dateEmprunt` = aujourd'hui ; `dateRetourPrevue` = aujourd'hui **+ 14 jours** ; `rendu` = `false` ; `dateRetourEffective` = `null`.
RM6	Un emprunt ne peut être rendu **qu'une seule fois**. Au retour : `rendu` = `true` ; `dateRetourEffective` = aujourd'hui ; le stock du livre **augmente de 1**.
RM7	Un emprunt est **en retard** s'il n'est **pas rendu et** si sa `dateRetourPrevue` est **strictement avant** aujourd'hui.
RM8	L'email de l'emprunteur est **enregistré en base**, mais **n'apparaît jamais** dans les réponses de l'API (donnée personnelle).

3. Modèle de données

3.1 Les trois tables



> Les flèches représentent un lien **logique** : `livres.auteur_id` contient l'id d'un auteur, `emprunts.livre_id` contient l'id d'un livre. Ce sont de simples colonnes numériques : aucune relation JPA n'est déclarée dans les entités. C'est le code des services qui vérifie que l'auteur ou le livre existe.
>
> Spring Boot transforme automatiquement le nom d'un attribut Java en nom de colonne : `anneePublication` → `annee_publication`, `auteurId` → `auteur_id`, `dateRetourPrevue` → `date_retour_prevue`...

3.2 Les attributs des entités

Auteur (classe `Auteur`, table `auteurs`)

Attribut Java	Type Java	Colonne obligatoire en base ?
`id`	`Long`	Oui (clé primaire générée par la base)
`nom`	`String`	Oui
`prenom`	`String`	Oui
`nationalite`	`String`	Non

Livre (classe `Livre`, table `livres`)

Attribut Java	Type Java	Colonne obligatoire en base ?
`id`	`Long`	Oui (clé primaire générée par la base)
`titre`	`String`	Oui
`isbn`	`String`	Oui, et **unique** (RM2)
`prix`	`Double`	Oui
`anneePublication`	`Integer`	Non
`stock`	`Integer`	Oui
`auteurId`	`Long`	Oui

Emprunt (classe `Emprunt`, table `emprunts`)

Attribut Java	Type Java	Colonne obligatoire en base ?
`id`	`Long`	Oui (clé primaire générée par la base)
`livreId`	`Long`	Oui
`emprunteur`	`String`	Oui
`emailEmprunteur`	`String`	Oui
`dateEmprunt`	`LocalDate`	Oui
`dateRetourPrevue`	`LocalDate`	Oui
`dateRetourEffective`	`LocalDate`	Non (`null` tant que le livre n'est pas rendu)
`rendu`	`Boolean`	Oui

> `LocalDate` (package `java.time`) représente une date sans heure. Méthodes utiles : `LocalDate.now()` (aujourd'hui), `date.plusDays(14)` (14 jours plus tard), `date1.isBefore(date2)` (vrai si `date1` est strictement avant `date2`). En JSON, une date s'écrit au format ISO 8601 : `"2026-11-02"`.

3.3 Les données initiales

Le fichier `data.sql` (fourni à la Q16) charge ces données à chaque démarrage. **Les identifiants sont toujours les mêmes** : la recette (§6) s'appuie dessus.

Auteurs

id	nom	prenom	nationalite	Remarque
1	Hugo	Victor	Française	
2	Rowling	J.K.	Britannique	
3	Camus	Albert	Française	
4	Tolkien	J.R.R.	Britannique	
5	Asimov	Isaac	Américaine	N'a **aucun** livre

Livres

id	titre	isbn	prix	anneePublication	stock	auteurId
1	Les Misérables	978-2-07-040850-4	12.50	1862	5	1
2	Notre-Dame de Paris	978-2-07-041239-6	9.90	1831	3	1
3	Harry Potter à l'école des sorciers	978-2-07-054127-0	8.90	1997	10	2
4	Harry Potter et la Chambre des secrets	978-2-07-054128-7	8.90	1998	7	2
5	L'Étranger	978-2-07-036024-8	6.50	1942	4	3
6	La Peste	978-2-07-036042-2	7.80	1947	6	3
7	Le Seigneur des Anneaux	978-2-267-02700-4	25.00	1954	3	4
8	Le Hobbit	978-2-267-02600-7	15.00	1937	**0**	4

Emprunts

id	livreId	emprunteur	emailEmprunteur	dateEmprunt	dateRetourPrevue	dateRetourEffective	rendu
1	1	Marie Dupont	marie.dupont@email.com	2026-09-01	2026-09-15	2026-09-10	`true`
2	3	Jean Martin	jean.martin@email.com	2026-09-05	2026-09-19	`null`	`false`
3	5	Sophie Bernard	sophie.bernard@email.com	2026-09-20	2026-10-04	`null`	`false`

> Les emprunts 2 et 3 ne sont pas rendus et leur date de retour prévue est dépassée : ils sont **en retard** (RM7).

4. Spécification de l'API

Cette section est la **référence** du projet. En cas de doute pendant une question, c'est ce qui est écrit ici qui fait foi.

4.1 Conventions communes à tous les endpoints

Adresse. Toutes les URLs commencent par `http://localhost:8080/api/v1`. Dans la suite, on écrit seulement la fin : `/livres` signifie `http://localhost:8080/api/v1/livres`.

Format. Les corps de requête et de réponse sont en JSON. Une réponse ne contient **jamais une entité** : uniquement un DTO de réponse (§4.3).

Pagination. Tous les endpoints qui renvoient une **liste** renvoient une page au format `PagedModel` (Partie 4 §2.2). Le client peut choisir `page`, `size` et `sort` dans l'URL. Sans paramètre, les valeurs par défaut sont :

Endpoints	Taille de page par défaut	Tri par défaut
Listes d'auteurs	`size = 10`	`nom`
Listes de livres	`size = 10`	`titre`
Listes d'emprunts	`size = 10`	`dateEmprunt`

Erreurs. Une erreur est signalée **uniquement par son code HTTP**. Les réponses d'erreur que vous construisez n'ont **pas de corps** (elles se terminent par `.build()`).

Erreurs automatiques. Spring produit seul certaines erreurs, **avant même** d'exécuter votre méthode. Vous n'avez **rien à coder** pour elles (et vous ne devez pas les modifier) :

Situation	Code produit par Spring
Le corps ne respecte pas les contraintes du DTO de requête (`@Valid`)	`400`
Le corps n'est pas du JSON valide	`400`
Un paramètre obligatoire (`@RequestParam`) est absent ou non numérique	`400`
Un identifiant de l'URL n'est pas un nombre (`/livres/abc`)	`400`
L'URL ne correspond à aucun endpoint	`404`

> Ces erreurs automatiques ont un petit corps JSON généré par Spring (`timestamp`, `status`, `error`, `path`). Il ne contient pas vos messages de validation : c'est normal, seul le code `400` est évalué.

Ordre des vérifications. Quand plusieurs erreurs sont possibles, la colonne « Erreurs » des tableaux §4.2 les donne **dans l'ordre où vous devez les vérifier**. La première vérification qui échoue détermine le code renvoyé. Les erreurs automatiques passent toujours en premier.

404 ou 400 ? Retenez la règle de ce projet :

- `404` : la ressource désignée par **l'URL** n'existe pas (exemple : `GET /livres/99`) ;
- `400` : une donnée **du corps ou des paramètres** est incorrecte, y compris un identifiant qui ne correspond à rien (exemple : `POST /livres` avec `"auteurId": 99`).

Fiche réflexe : un code HTTP = une instruction Java

Code	Signification dans ce projet	Instruction dans le contrôleur
`200`	Lecture ou modification réussie	`return ResponseEntity.ok(corps);`
`201`	Création réussie	`return ResponseEntity.status(HttpStatus.CREATED).body(corps);`
`204`	Suppression réussie (pas de corps)	`return ResponseEntity.noContent().build();`
`400`	Donnée incorrecte (hors validation automatique)	`return ResponseEntity.badRequest().build();`
`404`	Ressource de l'URL inexistante	`return ResponseEntity.notFound().build();`
`409`	Conflit avec une règle métier	`return ResponseEntity.status(HttpStatus.CONFLICT).build();`

4.2 Les 20 endpoints

Auteurs — classe `AuteurController`, préfixe `/auteurs`

N°	Méthode HTTP	URL	Méthode Java	Corps envoyé	Succès	Erreurs (dans l'ordre de vérification)
1	`GET`	`/auteurs`	`listerTous`	—	`200` + page d'`AuteurResponse`	—
2	`GET`	`/auteurs/{id}`	`obtenirParId`	—	`200` + `AuteurResponse`	`404` auteur inexistant
3	`POST`	`/auteurs`	`creer`	`AuteurRequest`	`201` + `AuteurResponse`	`400` validation (automatique)
4	`PUT`	`/auteurs/{id}`	`mettreAJour`	`AuteurRequest`	`200` + `AuteurResponse`	`400` validation (automatique) → `404` auteur inexistant
5	`DELETE`	`/auteurs/{id}`	`supprimer`	—	`204` sans corps	`404` auteur inexistant → `409` l'auteur a encore des livres (RM3)
6	`GET`	`/auteurs/{id}/livres`	`listerLivres`	—	`200` + page de `LivreResponse`	`404` auteur inexistant

Livres — classe `LivreController`, préfixe `/livres`

N°	Méthode HTTP	URL	Méthode Java	Corps envoyé	Succès	Erreurs (dans l'ordre de vérification)
7	`GET`	`/livres`	`listerTous`	—	`200` + page de `LivreResponse`	—
8	`GET`	`/livres/{id}`	`obtenirParId`	—	`200` + `LivreResponse`	`404` livre inexistant
9	`POST`	`/livres`	`creer`	`LivreRequest`	`201` + `LivreResponse`	`400` validation (automatique) → `400` auteur inexistant → `409` ISBN déjà utilisé (RM2)
10	`PUT`	`/livres/{id}`	`mettreAJour`	`LivreRequest`	`200` + `LivreResponse`	`400` validation (automatique) → `404` livre inexistant → `400` auteur inexistant → `409` ISBN déjà utilisé par **un autre** livre (RM2)
11	`DELETE`	`/livres/{id}`	`supprimer`	—	`204` sans corps	`404` livre inexistant
12	`GET`	`/livres/recherche?titre=...`	`rechercher`	—	`200` + page de `LivreResponse` (éventuellement vide)	`400` paramètre `titre` absent (automatique)
13	`GET`	`/livres/disponibles`	`listerDisponibles`	—	`200` + page de `LivreResponse` (RM4)	—
14	`PATCH`	`/livres/{id}/stock?quantite=...`	`ajouterStock`	—	`200` + `LivreResponse`	`400` paramètre `quantite` absent ou non numérique (automatique) → `400` `quantite` ≤ 0 → `404` livre inexistant

> **N° 12 — recherche.** Le texte est cherché **n'importe où** dans le titre, **sans tenir compte des majuscules** : `?titre=harry` trouve « Harry Potter à l'école des sorciers ».
> **N° 14 — réapprovisionnement.** `quantite` est le nombre d'exemplaires **ajoutés** au stock actuel : un stock de 3 avec `?quantite=2` devient 5.

Emprunts — classe `EmpruntController`, préfixe `/emprunts`

N°	Méthode HTTP	URL	Méthode Java	Corps envoyé	Succès	Erreurs (dans l'ordre de vérification)
15	`GET`	`/emprunts`	`listerTous`	—	`200` + page d'`EmpruntResponse`	—
16	`GET`	`/emprunts/{id}`	`obtenirParId`	—	`200` + `EmpruntResponse`	`404` emprunt inexistant
17	`POST`	`/emprunts`	`creer`	`EmpruntRequest`	`201` + `EmpruntResponse` (RM5)	`400` validation (automatique) → `400` livre inexistant → `409` livre indisponible (RM4)
18	`PATCH`	`/emprunts/{id}/retour`	`retourner`	—	`200` + `EmpruntResponse` (RM6)	`404` emprunt inexistant → `409` emprunt déjà rendu (RM6)
19	`GET`	`/emprunts/en-cours`	`listerEnCours`	—	`200` + page des emprunts **non rendus**	—
20	`GET`	`/emprunts/en-retard`	`listerEnRetard`	—	`200` + page des emprunts **en retard** (RM7)	—

4.3 Les DTOs

Les DTOs de **requête** décrivent ce que le client envoie ; ceux de **réponse**, ce qu'il reçoit (Partie 4 §2.1).

DTOs de requête : attributs, contraintes de validation et messages

Chaque contrainte s'écrit avec l'annotation indiquée et **exactement** le message indiqué.

`AuteurRequest`

Attribut	Type	Annotations et messages
`nom`	`String`	`@NotBlank` « Le nom est obligatoire » ; `@Size(max = 100)` « Le nom ne doit pas dépasser 100 caractères »
`prenom`	`String`	`@NotBlank` « Le prénom est obligatoire » ; `@Size(max = 100)` « Le prénom ne doit pas dépasser 100 caractères »
`nationalite`	`String`	Optionnel. `@Size(max = 50)` « La nationalité ne doit pas dépasser 50 caractères »

`LivreRequest`

Attribut	Type	Annotations et messages
`titre`	`String`	`@NotBlank` « Le titre est obligatoire » ; `@Size(max = 200)` « Le titre ne doit pas dépasser 200 caractères »
`isbn`	`String`	`@NotBlank` « L'ISBN est obligatoire » ; `@Size(max = 20)` « L'ISBN ne doit pas dépasser 20 caractères »
`prix`	`Double`	`@NotNull` « Le prix est obligatoire » ; `@Positive` « Le prix doit être positif »
`anneePublication`	`Integer`	Optionnel (pas de `@NotNull`). `@Positive` « L'année de publication doit être positive »
`stock`	`Integer`	`@NotNull` « Le stock est obligatoire » ; `@PositiveOrZero` « Le stock ne peut pas être négatif »
`auteurId`	`Long`	`@NotNull` « L'identifiant de l'auteur est obligatoire »

`EmpruntRequest`

Attribut	Type	Annotations et messages
`livreId`	`Long`	`@NotNull` « L'identifiant du livre est obligatoire »
`emprunteur`	`String`	`@NotBlank` « Le nom de l'emprunteur est obligatoire » ; `@Size(max = 100)` « Le nom de l'emprunteur ne doit pas dépasser 100 caractères »
`emailEmprunteur`	`String`	`@NotBlank` « L'email est obligatoire » ; `@Email` « L'email doit être valide »

> Écriture d'une annotation avec son message : `@Size(max = 100, message = "Le nom ne doit pas dépasser 100 caractères")`.
> `@Positive` accepte la valeur `null` : c'est pourquoi `prix` porte aussi `@NotNull`, alors que `anneePublication`, optionnelle, n'a que `@Positive` (Partie 4 §2.1).

DTOs de réponse : attributs, dans l'ordre d'affichage JSON

DTO	Attributs, dans cet ordre (type)
`AuteurResponse`	`id` (`Long`), `nom` (`String`), `prenom` (`String`), `nationalite` (`String`)
`LivreResponse`	`id` (`Long`), `titre` (`String`), `isbn` (`String`), `prix` (`Double`), `anneePublication` (`Integer`), `stock` (`Integer`), `auteurId` (`Long`)
`EmpruntResponse`	`id` (`Long`), `livreId` (`Long`), `emprunteur` (`String`), `dateEmprunt` (`LocalDate`), `dateRetourPrevue` (`LocalDate`), `dateRetourEffective` (`LocalDate`), `rendu` (`Boolean`), `enRetard` (`Boolean`)

> `EmpruntResponse` ne contient **pas** `emailEmprunteur` (RM8). Le champ `enRetard` n'existe pas en base : il est **calculé** par le mapper avec la règle RM7 (Q27).

4.4 Exemples de réponses JSON

`GET /auteurs/1` → `200`

```
{
  "id": 1,
  "nom": "Hugo",
  "prenom": "Victor",
  "nationalite": "Française"
}
```

`GET /livres/7` → `200`

```
{
  "id": 7,
  "titre": "Le Seigneur des Anneaux",
  "isbn": "978-2-267-02700-4",
  "prix": 25.0,
  "anneePublication": 1954,
  "stock": 3,
  "auteurId": 4
}
```

> Un `Double` est écrit sans zéro inutile : `25.00` en base devient `25.0` en JSON, `12.50` devient `12.5`.

`GET /emprunts/2` → `200`

```
{
  "id": 2,
  "livreId": 3,
  "emprunteur": "Jean Martin",
  "dateEmprunt": "2026-09-05",
  "dateRetourPrevue": "2026-09-19",
  "dateRetourEffective": null,
  "rendu": false,
  "enRetard": true
}
```

`GET /auteurs?size=2` → `200` (une page)

```
{
  "content": [
    { "id": 5, "nom": "Asimov", "prenom": "Isaac", "nationalite": "Américaine" },
    { "id": 3, "nom": "Camus", "prenom": "Albert", "nationalite": "Française" }
  ],
  "page": {
    "size": 2,
    "number": 0,
    "totalElements": 5,
    "totalPages": 3
  }
}
```

> `number` est le numéro de la page **en commençant à 0**. L'ordre des champs à l'intérieur de `"page"` peut varier selon la version de Jackson : ce n'est pas une erreur.

5. Les questions

> **Comment lire une question.** Chaque question indique : le ou les **fichiers** à créer, ce qu'il faut **faire**, les **règles** à respecter, puis comment **vérifier** votre travail. Faites les questions **dans l'ordre** : chacune s'appuie sur les précédentes.

Partie A — Conception (Sprint 1, 40 min)

Répondez dans un document (Word, LibreOffice, Google Docs...) que vous exporterez en PDF sous le nom `CONCEPTION.pdf` (vous le placerez à la racine du projet à la Q10). Pour chaque question, écrivez son numéro en titre (Q1, Q2...) puis votre réponse. Appuyez-vous sur les §2 à §4 de cet énoncé et sur les parties du cours indiquées.

Q1 — Ressources et URLs (Partie 2 §2.3 et §2.4)

1. Citez les trois ressources de l'API.
2. Pour chaque ressource, écrivez l'URL complète de la **collection** et l'URL complète de l'**élément d'id 3**.
3. Citez **quatre** règles de nommage REST respectées par ces URLs. Pour chaque règle, donnez une URL **incorrecte** qui la viole (exemple de forme attendue : « Pas de verbe dans l'URL — incorrect : `/api/v1/getLivres` »).

Q2 — Méthodes HTTP (Partie 2 §2.5)

1. Pourquoi la création d'un emprunt utilise-t-elle `POST /emprunts` et non `PUT /emprunts` ?
2. Pourquoi le retour d'un emprunt (`/emprunts/{id}/retour`) et le réapprovisionnement (`/livres/{id}/stock`) utilisent-ils `PATCH` et non `PUT` ?
3. Pourquoi le titre recherché est-il passé en **paramètre de requête** (`/livres/recherche?titre=harry`) et non dans le chemin (`/livres/recherche/harry`) ?

Q3 — Idempotence (Partie 1 §2.7)

On part des données initiales (§3.3).

1. On envoie **deux fois de suite** la même requête `POST /emprunts` avec le corps `{"livreId": 1, "emprunteur": "Alice Durand", "emailEmprunteur": "alice.durand@email.com"}`. Donnez le code HTTP de chaque envoi, le nombre d'emprunts créés et le stock final du livre 1.
2. On envoie **deux fois de suite** `DELETE /livres/6`. Donnez le code HTTP de chaque envoi et l'état final de la base pour le livre 6.
3. Laquelle de ces deux méthodes est idempotente ? Justifiez en une phrase à partir de vos réponses 1 et 2.

Q4 — Codes de statut (Partie 2 §2.6 et §4.1 de l'énoncé)

1. Recopiez et complétez ce tableau (une seule réponse possible par ligne) :

#	Situation	Code HTTP
a	`POST /auteurs` avec un corps valide	
b	`DELETE /livres/2` (le livre existe)	
c	`GET /livres/999`	
d	`POST /livres` avec un corps valide mais `"auteurId": 999`	
e	`POST /livres` avec l'ISBN du livre 1	
f	`POST /emprunts` pour le livre 8 (« Le Hobbit »)	
g	`POST /auteurs` avec `"nom": ""`	
h	`DELETE /auteurs/1` (Victor Hugo)	

2. Expliquez pourquoi la situation c renvoie `404` alors que la situation d renvoie `400`.
3. Pourquoi ne faut-il **jamais** répondre `200 OK` avec un corps du type `{"erreur": "Livre introuvable"}` ?

Q5 — Représentations JSON (Partie 1 §2.6, Partie 4 §2.1)

1. Écrivez la réponse JSON **complète** de `GET /livres/1` avec les données initiales (respectez l'ordre des champs du §4.3).
2. Le 2 novembre 2026, sur une API fraîchement démarrée, on envoie `POST /emprunts` avec le corps `{"livreId": 2, "emprunteur": "Alice Durand", "emailEmprunteur": "alice.durand@email.com"}`. Écrivez le code HTTP et la réponse JSON **complète** (l'emprunt reçoit l'id 4). Indiquez ensuite le nouveau stock du livre 2.

Q6 — Pagination (Partie 4 §2.2)

1. Écrivez l'URL complète qui demande la **deuxième** page des livres, avec **3** livres par page, triés par **prix décroissant**.
2. Avec les 8 livres initiaux et 3 livres par page, quelles sont les valeurs de `page.totalElements` et de `page.totalPages` ?
3. Donnez deux raisons de paginer une collection plutôt que de tout renvoyer.

Q7 — Sans état (Partie 2 §2.2)

1. Que signifie la contrainte REST « sans état » (*stateless*) ?
2. Le corps de `POST /emprunts` contient le nom et l'email de l'emprunteur. Expliquez le lien avec la contrainte « sans état ».

Q8 — DTOs (Partie 4 §2.1)

1. Pourquoi `EmpruntRequest` ne contient-il ni `id`, ni dates, ni `rendu` ?
2. Pourquoi `EmpruntResponse` ne contient-il pas `emailEmprunteur` ?

3. Donnez deux problèmes que poserait le fait de renvoyer directement l'entité `Emprunt` au client.

Q9 — Tester avec curl (Partie 1 §3.2 et §3.4)

1. Écrivez la commande curl qui affiche le **code de statut et les en-têtes** de la réponse de `GET /auteurs/1`.
2. Quel en-tête de la réponse indique que le corps est du JSON ? Quelle valeur doit-il avoir ?
3. Écrivez la commande curl complète qui crée l'auteur `{"nom": "Verne", "prenom": "Jules", "nationalite": "Française"}`. Expliquez le rôle de chaque option utilisée.

Partie B — Initialiser le projet (Sprint 1, 20 min)

Q10 — Générer le projet Spring Boot (Partie 3 §3.1 et §3.2)

Faire. Sur https://start.spring.io, générez le projet avec exactement ces paramètres :

Paramètre	Valeur
Project	Maven
Language	Java
Spring Boot	4.1.1
Group	`com.formation`
Artifact	`bibliotheque-api`
Name	`bibliotheque-api`
Package name	`com.formation.bibliotheque`
Packaging	Jar
Java	25
Dependencies	Spring Web, Spring Data JPA, H2 Database, Validation, Spring Boot DevTools

Décompressez l'archive, ouvrez le projet dans votre IDE, puis **remplacez tout le bloc `<dependencies>`** du `pom.xml` par celui-ci (il ajoute la console H2, la documentation Swagger et les starters de test) :

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
        <version>3.1.1</version>
    </dependency>
    <dependency>
```

```
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-h2console</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Placez votre fichier `CONCEPTION.pdf` à la racine du projet (à côté du `pom.xml`).

Le fichier de test généré (`src/test/java/.../BibliothequeApplicationTests.java`) reste tel quel : vous n'avez aucun test à écrire.

Vérifier. Lancez la classe `BibliothequeApplication` (bouton ▶ de l'IDE, ou `./mvnw spring-boot:run` sous Linux/macOS, `mvnw.cmd spring-boot:run` sous Windows). La console affiche `Tomcat started on port 8080`. L'adresse `http://localhost:8080` affiche une page d'erreur 404 : c'est normal, aucun contrôleur n'existe encore.

Q11 — Créer les packages et la configuration (Partie 3 §3.3 et §3.4)

Faire.

1. Dans `src/main/java/com/formation/bibliotheque/`, créez les sept packages : `config`, `controller`, `dto`, `mapper`, `model`, `repository`, `service`.
2. Remplacez **tout** le contenu de `src/main/resources/application.properties` par :

```
spring.application.name=bibliotheque-api
server.port=8080

# Base H2 en mémoire
spring.datasource.url=jdbc:h2:mem:bibliothequedb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# Console H2 : http://localhost:8080/h2-console
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
# Exécuter data.sql APRÈS la création des tables
spring.jpa.defer-datasource-initialization=true

# Swagger UI : http://localhost:8080/swagger-ui.html
springdoc.swagger-ui.path=/swagger-ui.html
```

Vérifier. L'application démarre toujours sans erreur.

Partie C — Entités, repositories et données (Sprint 1, 45 min)

Règles communes aux trois entités (Q12 à Q14) — Partie 3 §3.6 :

- la classe porte `@Entity` et `@Table(name = "...")` avec le nom de table indiqué ;
- l'attribut `id` porte `@Id` et `@GeneratedValue(strategy = GenerationType.IDENTITY)` ;
- chaque attribut **obligatoire en base** (tableaux du §3.2) porte `@Column(nullable = false)` ; un attribut non obligatoire ne porte aucune annotation ;
- la classe a un **constructeur vide** (obligatoire pour JPA) et un **getter et un setter pour chaque attribut** ;
- **aucune** annotation de validation (`@NotBlank`, `@Positive`...) dans les entités : la validation est faite par les DTOs de requête (Partie 4 §2.1).

> Les annotations JPA (`@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `GenerationType`, `@Column`) viennent du package `jakarta.persistence`.

Q12 — L'entité `Auteur`

Fichier. `model/Auteur.java` — table `auteurs`.

Faire. Écrivez la classe avec les attributs du §3.2 (Auteur) en appliquant les règles communes.

Q13 — L'entité `Livre`

Fichier. `model/Livre.java` — table `livres`.

Faire. Écrivez la classe avec les attributs du §3.2 (Livre) en appliquant les règles communes.

Règle supplémentaire. L'ISBN est unique (RM2) : son annotation est `@Column(nullable = false, unique = true)`.

Q14 — L'entité `Emprunt`

Fichier. `model/Emprunt.java` — table `emprunts`.

Faire. Écrivez la classe avec les attributs du §3.2 (Emprunt) en appliquant les règles communes. Les dates sont de type `java.time.LocalDate`.

Q15 — Les trois repositories (Partie 3 §3.7)

Fichiers. `repository/AuteurRepository.java`, `repository/LivreRepository.java`, `repository/EmpruntRepository.java`.

Faire. Chaque repository est une **interface** annotée `@Repository` qui hérite de `JpaRepository<Entité, Long>`. Elle hérite ainsi de `findAll(Pageable)`, `findById`, `existsById`, `save` et `deleteById`. Ajoutez **exactement** les méthodes ci-dessous : Spring Data écrit les requêtes SQL à partir de leur nom.

```
public interface AuteurRepository extends JpaRepository<Auteur, Long> {
    // aucune méthode à ajouter
}
```

```
public interface LivreRepository extends JpaRepository<Livre, Long> {

    Optional<Livre> findByIsbn(String isbn);                                    // le livre qui a cet ISBN

    boolean existsByAuteurId(Long auteurId);                                    // true si l'auteur a au moins
un livre

    Page<Livre> findByAuteurId(Long auteurId, Pageable pageable);               // les livres d'un auteur

    Page<Livre> findByTitreContainingIgnoreCase(String titre, Pageable pageable); // titre contenant le texte

    Page<Livre> findByStockGreaterThan(Integer stock, Pageable pageable);       // stock strictement supérieur
}

public interface EmpruntRepository extends JpaRepository<Emprunt, Long> {

    Page<Emprunt> findByRenduFalse(Pageable pageable);                          // emprunts non rendus

    Page<Emprunt> findByRenduFalseAndDateRetourPrevueBefore(LocalDate date,     // non rendus ET date prévue
                                                            Pageable pageable); // strictement avant "date"
}
```

> `Page` et `Pageable` viennent du package `org.springframework.data.domain`. Attention : l'IDE propose aussi `java.awt.print.Pageable`, qui n'a rien à voir avec Spring.

Q16 — Les données initiales (Partie 3 §3.10)

Fichier. `src/main/resources/data.sql`.

Faire. Copiez ce script **sans le modifier** :

```
-- Auteurs (id 1 à 5). Isaac Asimov (id 5) n'a aucun livre.
INSERT INTO auteurs (nom, prenom, nationalite) VALUES
('Hugo', 'Victor', 'Française'),
('Rowling', 'J.K.', 'Britannique'),
('Camus', 'Albert', 'Française'),
('Tolkien', 'J.R.R.', 'Britannique'),
('Asimov', 'Isaac', 'Américaine');


-- Livres (id 1 à 8). Le Hobbit (id 8) a un stock de 0 : il n'est pas disponible.
INSERT INTO livres (titre, isbn, prix, annee_publication, stock, auteur_id) VALUES
('Les Misérables', '978-2-07-040850-4', 12.50, 1862, 5, 1),
('Notre-Dame de Paris', '978-2-07-041239-6', 9.90, 1831, 3, 1),
('Harry Potter à l''école des sorciers', '978-2-07-054127-0', 8.90, 1997, 10, 2),
('Harry Potter et la Chambre des secrets', '978-2-07-054128-7', 8.90, 1998, 7, 2),
('L''Étranger', '978-2-07-036024-8', 6.50, 1942, 4, 3),
('La Peste', '978-2-07-036042-2', 7.80, 1947, 6, 3),
('Le Seigneur des Anneaux', '978-2-267-02700-4', 25.00, 1954, 3, 4),
('Le Hobbit', '978-2-267-02600-7', 15.00, 1937, 0, 4);


-- Emprunts (id 1 à 3). L'emprunt 1 est rendu ; les emprunts 2 et 3 sont en retard.
INSERT INTO emprunts (livre_id, emprunteur, email_emprunteur, date_emprunt, date_retour_prevue,
date_retour_effective, rendu) VALUES
(1, 'Marie Dupont', 'marie.dupont@email.com', '2026-09-01', '2026-09-15', '2026-09-10', TRUE),
(3, 'Jean Martin', 'jean.martin@email.com', '2026-09-05', '2026-09-19', NULL, FALSE),
(5, 'Sophie Bernard', 'sophie.bernard@email.com', '2026-09-20', '2026-10-04', NULL, FALSE);
```

Vérifier. Redémarrez l'application, ouvrez `http://localhost:8080/h2-console` (JDBC URL : `jdbc:h2:mem:bibliothequedb`, User Name : `sa`, mot de passe vide) et exécutez :

```
SELECT COUNT(*) FROM auteurs;   -- résultat attendu : 5
SELECT COUNT(*) FROM livres;    -- résultat attendu : 8
SELECT COUNT(*) FROM emprunts;  -- résultat attendu : 3
```

> Si l'application refuse de démarrer avec une erreur SQL, comparez les noms d'attributs de vos entités avec le §3.2 : une colonne du script n'existe pas dans votre table.

Partie D — Les auteurs, de bout en bout (Sprint 1, 1 h 15)

Q17 — Les DTOs `AuteurRequest` et `AuteurResponse` (Partie 4 §2.1)

Fichiers. `dto/AuteurRequest.java`, `dto/AuteurResponse.java`.

Faire.

1. `AuteurRequest` : les attributs `private` et les annotations de validation du §4.3, un **constructeur vide**, un getter et un setter par attribut.
2. `AuteurResponse` : les attributs du §4.3, déclarés `private final` ; **un seul constructeur** qui reçoit tous les attributs **dans l'ordre du tableau** ; un getter par attribut ; **aucun setter**. Placez au-dessus de la classe :

```
@JsonPropertyOrder({"id", "nom", "prenom", "nationalite"})
```

> Les annotations de validation (`@NotBlank`, `@Size`, `@NotNull`, `@Positive`, `@PositiveOrZero`, `@Email`) viennent du package `jakarta.validation.constraints`.
>
> `@JsonPropertyOrder` vient du package `com.fasterxml.jackson.annotation`. Sans elle, Jackson 3 trie les champs par ordre alphabétique (Partie 4 §2.1).

Q18 — Le mapper `AuteurMapper` (Partie 4 §2.1 et §2.2)

Fichier. `mapper/AuteurMapper.java`.

Faire. Une classe `public final class AuteurMapper` avec un constructeur `private` vide (on n'en crée jamais d'instance) et **quatre méthodes `public static`** :

Signature	Ce que fait la méthode
`Auteur versEntite(AuteurRequest request)`	Crée un `Auteur` et y recopie `nom`, `prenom`, `nationalite`. L'`id` reste `null`.
`AuteurResponse versResponse(Auteur auteur)`	Crée un `AuteurResponse` à partir des attributs de l'auteur.
`List<AuteurResponse> versResponses(List<Auteur> auteurs)`	Parcourt la liste avec une boucle `for` et convertit chaque auteur avec `versResponse`.
`PagedModel<AuteurResponse> versPage(Page<Auteur> page)`	Convertit une page d'entités en page de DTOs. Code fourni ci-dessous.

```
public static PagedModel<AuteurResponse> versPage(Page<Auteur> page) {
    // 1. On convertit le contenu de la page en DTOs
    List<AuteurResponse> contenu = versResponses(page.getContent());
    // 2. On reconstruit une page de DTOs avec les mêmes informations de pagination
    Page<AuteurResponse> pageDto = new PageImpl<>(contenu, page.getPageable(), page.getTotalElements());
    // 3. PagedModel produit un JSON stable : { "content": [...], "page": {...} }
    return new PagedModel<>(pageDto);
}
```

> Imports : `Page` et `PageImpl` (`org.springframework.data.domain`), `PagedModel` (`org.springframework.data.web`).

Q19 — Le service `AuteurService` (Partie 3 §3.8)

Fichier. `service/AuteurService.java`.

Faire. Recopiez ce début de classe **tel quel**, puis écrivez les méthodes du tableau.

```
@Service
public class AuteurService {


    private final AuteurRepository auteurRepository;
    private final LivreRepository livreRepository;


    public AuteurService(AuteurRepository auteurRepository, LivreRepository livreRepository) {
        this.auteurRepository = auteurRepository;
        this.livreRepository = livreRepository;
    }


    // Écrivez ici les méthodes du tableau
}
```

Signature	Ce que fait la méthode
`Page<Auteur> recupererTous(Pageable pageable)`	Renvoie une page d'auteurs (`findAll(pageable)`).
`Optional<Auteur> recupererParId(Long id)`	Renvoie l'auteur, ou un `Optional` vide s'il n'existe pas.
`boolean existe(Long id)`	Renvoie `true` si un auteur porte cet id.
`Auteur creer(Auteur auteur)`	Met l'`id` à `null` (pour forcer une création), sauvegarde et renvoie l'auteur sauvegardé.
`Auteur mettreAJour(Auteur auteurExistant, Auteur nouvellesValeurs)`	Recopie `nom`, `prenom` et `nationalite` de `nouvellesValeurs` dans `auteurExistant`, sauvegarde `auteurExistant` et le renvoie.
`boolean aDesLivres(Long id)`	**RM3** : renvoie `true` si au moins un livre a cet auteur (`livreRepository.existsByAuteurId`).
`void supprimer(Long id)`	Supprime l'auteur. Les vérifications ont été faites avant, par le contrôleur.

> Le service ne connaît pas HTTP : il ne renvoie **jamais** de `ResponseEntity` ni de code de statut. C'est le contrôleur qui traduit ses résultats en codes HTTP.

Q20 — Le contrôleur `AuteurController` : endpoints 1 à 5 (Partie 3 §3.9)

Fichier. `controller/AuteurController.java`.

Faire. Recopiez ce début de classe **tel quel**, puis écrivez les méthodes Java des endpoints **1 à 5** (§4.2). L'endpoint 6 sera ajouté à la Q25.

```
@RestController
@RequestMapping("/api/v1/auteurs")
public class AuteurController {


    private final AuteurService auteurService;


    public AuteurController(AuteurService auteurService) {
        this.auteurService = auteurService;
    }


    // Écrivez ici les méthodes des endpoints 1 à 5
}
```

Règles. Chaque méthode suit le **même plan en trois temps** :

1. **vérifier**, dans l'ordre de la colonne « Erreurs » du §4.2 : chaque vérification qui échoue renvoie immédiatement son code (fiche réflexe du §4.1) ;
2. **agir** : appeler le service, en convertissant le DTO de requête en entité avec le mapper ;

3. **répondre** : renvoyer le code de succès avec un DTO de réponse, jamais l'entité.

Les endpoints qui reçoivent un corps déclarent leur paramètre `@Valid @RequestBody AuteurRequest request`. Les endpoints qui ont `{id}` dans l'URL déclarent `@PathVariable Long id`. Les endpoints de liste déclarent `@PageableDefault(size = 10, sort = "nom") Pageable pageable`.

Deux méthodes vous sont données comme modèles :

```
// Endpoint 1 — GET /api/v1/auteurs : une liste paginée
@GetMapping
public ResponseEntity<PagedModel<AuteurResponse>> listerTous(
        @PageableDefault(size = 10, sort = "nom") Pageable pageable) {
    Page<Auteur> page = auteurService.recupererTous(pageable);
    return ResponseEntity.ok(AuteurMapper.versPage(page));                  // 200
}

// Endpoint 5 — DELETE /api/v1/auteurs/{id} : deux vérifications, puis l'action
@DeleteMapping("/{id}")
public ResponseEntity<Void> supprimer(@PathVariable Long id) {
    // 1. Vérifier, dans l'ordre du tableau
    if (!auteurService.existe(id)) {
        return ResponseEntity.notFound().build();                           // 404
    }
    if (auteurService.aDesLivres(id)) {
        return ResponseEntity.status(HttpStatus.CONFLICT).build();          // 409 (RM3)
    }
    // 2. Agir
    auteurService.supprimer(id);
    // 3. Répondre
    return ResponseEntity.noContent().build();                              // 204
}
```

> Pour lire un `Optional` sans lambda (endpoints 2 et 4) : `Optional<Auteur> resultat = auteurService.recupererParId(id);` puis `if (resultat.isEmpty()) { return ResponseEntity.notFound().build(); }` et enfin `resultat.get()` pour obtenir l'auteur.

Imports utiles pour les trois contrôleurs

Classe ou annotation	Package
`@RestController`, `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, `@DeleteMapping`, `@PathVariable`, `@RequestBody`, `@RequestParam`	`org.springframework.web.bind.annotation`
`ResponseEntity`, `HttpStatus`	`org.springframework.http`
`Page`, `Pageable`	`org.springframework.data.domain`
`PageableDefault`, `PagedModel`	`org.springframework.data.web`
`@Valid`	`jakarta.validation`
`Optional`	`java.util`

Vérifier. Redémarrez l'application, puis exécutez **dans l'ordre** les requêtes **R01 à R10** du §6 dans Postman, en les enregistrant dans votre collection (Q32). Chaque code obtenu doit être égal au code attendu.

> **Fin du sprint 1.** Démonstration : `CONCEPTION.pdf` rempli et requêtes R01 à R10 conformes.

Partie E — Les livres (Sprint 2, 1 h 15)

Q21 — Les DTOs `LivreRequest` et `LivreResponse`

Fichiers. `dto/LivreRequest.java`, `dto/LivreResponse.java`.

Faire. Mêmes règles qu'à la Q17, avec les attributs et contraintes du §4.3. Pour `LivreResponse` :

```
@JsonPropertyOrder({"id", "titre", "isbn", "prix", "anneePublication", "stock", "auteurId"})
```

Q22 — Le mapper `LivreMapper`

Fichier. `mapper/LivreMapper.java`.

Faire. Mêmes règles qu'à la Q18 : `versEntite` (recopie les six attributs de la requête), `versResponse`, `versResponses` et `versPage` pour le type `Livre`.

Q23 — Le service `LivreService`

Fichier. `service/LivreService.java`.

Faire. Recopiez ce début de classe **tel quel**, puis écrivez les méthodes du tableau.

```
@Service
public class LivreService {

    private final LivreRepository livreRepository;
    private final AuteurRepository auteurRepository;

    public LivreService(LivreRepository livreRepository, AuteurRepository auteurRepository) {
        this.livreRepository = livreRepository;
        this.auteurRepository = auteurRepository;
    }

    // Écrivez ici les méthodes du tableau
}
```

Signature	Ce que fait la méthode
`Page<Livre> recupererTous(Pageable pageable)`	Renvoie une page de livres.
`Optional<Livre> recupererParId(Long id)`	Renvoie le livre, ou un `Optional` vide.
`boolean existe(Long id)`	Renvoie `true` si un livre porte cet id.
`boolean auteurExiste(Long auteurId)`	Renvoie `true` si un auteur porte cet id (`auteurRepository.existsById`).
`boolean isbnExiste(String isbn)`	**RM2, création** : renvoie `true` si un livre porte déjà cet ISBN (`findByIsbn` puis `isPresent()`).
`boolean isbnUtiliseParUnAutreLivre(String isbn, Long idLivre)`	**RM2, modification** : renvoie `true` si cet ISBN appartient à un livre **dont l'id est différent** de `idLivre`. Si aucun livre n'a cet ISBN, renvoie `false`. Si c'est le livre `idLivre` lui-même, renvoie `false` (un livre peut garder son ISBN).
`Livre creer(Livre livre)`	Met l'`id` à `null`, sauvegarde et renvoie le livre sauvegardé.
`Livre mettreAJour(Livre livreExistant, Livre nouvellesValeurs)`	Recopie `titre`, `isbn`, `prix`, `anneePublication`, `stock` et `auteurId` dans `livreExistant`, le sauvegarde et le renvoie.
`void supprimer(Long id)`	Supprime le livre (les vérifications ont été faites avant).
`Page<Livre> rechercherParTitre(String titre, Pageable pageable)`	Renvoie les livres dont le titre contient `titre`, sans tenir compte des majuscules.

Signature	Ce que fait la méthode
`Page<Livre> recupererDisponibles(Pageable pageable)`	**RM4** : renvoie les livres dont le stock est strictement supérieur à `0`.
`Livre ajouterStock(Livre livre, Integer quantite)`	Ajoute `quantite` au stock actuel du livre, le sauvegarde et le renvoie.

> Pour comparer deux `Long`, utilisez `equals` et non `==` : `livreTrouve.getId().equals(idLivre)`.

Q24 — Le contrôleur `LivreController` : endpoints 7 à 14

Fichier. `controller/LivreController.java`.

Faire. Recopiez ce début de classe **tel quel**, puis écrivez les méthodes Java des endpoints **7 à 14** en suivant le plan en trois temps de la Q20.

```
@RestController
@RequestMapping("/api/v1/livres")
public class LivreController {

    private final LivreService livreService;

    public LivreController(LivreService livreService) {
        this.livreService = livreService;
    }

    // Écrivez ici les méthodes des endpoints 7 à 14
}
```

Règles.

- Listes : `@PageableDefault(size = 10, sort = "titre") Pageable pageable`.
- Endpoint 12 : le titre est lu avec `@RequestParam String titre` (obligatoire : s'il est absent, Spring renvoie `400` tout seul).
- Endpoint 14 : l'annotation est `@PatchMapping("/{id}/stock")` et la quantité est lue avec `@RequestParam Integer quantite`.
- Endpoint 10 : l'ordre des vérifications est important. Si le livre n'existe pas, la réponse est `404`, même si l'ISBN envoyé est celui d'un autre livre.
- Les URLs `/recherche` et `/disponibles` ne posent pas de conflit avec `/{id}` : Spring choisit toujours le chemin écrit en toutes lettres.

Vérifier. Redémarrez l'application, puis exécutez dans l'ordre **R01 à R10**, puis **R13 à R28**. Tous les codes doivent être conformes.

Q25 — Compléter les auteurs : endpoint 6, les livres d'un auteur

Fichiers. `service/AuteurService.java`, `controller/AuteurController.java`.

Faire.

1. Ajoutez à `AuteurService` la méthode `Page<Livre> recupererLivres(Long id, Pageable pageable)`, qui renvoie les livres de l'auteur (`livreRepository.findByAuteurId`).
2. Ajoutez à `AuteurController` la méthode `listerLivres` de l'endpoint 6 : `@GetMapping("/{id}/livres")`, paramètres `@PathVariable Long id` et

`@PageableDefault(size = 10, sort = "titre") Pageable pageable`. Elle renvoie `404` si l'auteur n'existe pas, sinon `200` avec `LivreMapper.versPage(...)`.

Vérifier. Redémarrez l'application, puis exécutez dans l'ordre **R01 à R28**. Tous les codes doivent être conformes.

Partie F — Les emprunts (Sprint 2, 1 h 15)

Q26 — Les DTOs `EmpruntRequest` et `EmpruntResponse`

Fichiers. `dto/EmpruntRequest.java`, `dto/EmpruntResponse.java`.

Faire. Mêmes règles qu'à la Q17, avec les attributs et contraintes du §4.3. Pour `EmpruntResponse` :

```
@JsonPropertyOrder({"id", "livreId", "emprunteur", "dateEmprunt", "dateRetourPrevue",
        "dateRetourEffective", "rendu", "enRetard"})
```

Q27 — Le mapper `EmpruntMapper`

Fichier. `mapper/EmpruntMapper.java`.

Faire. Mêmes règles qu'à la Q18, avec deux différences :

Méthode	Différence
`Emprunt versEntite(EmpruntRequest request)`	Recopie **uniquement** `livreId`, `emprunteur` et `emailEmprunteur`. Les dates et `rendu` restent vides : c'est le service qui les remplit (RM5).
`EmpruntResponse versResponse(Emprunt emprunt)`	Ne recopie **pas** l'email (RM8). Calcule `enRetard` avec la règle **RM7** : `true` si `rendu` vaut `false` **et** si `dateRetourPrevue` est avant `LocalDate.now()` ; sinon `false`.

Q28 — Le service `EmpruntService`

Fichier. `service/EmpruntService.java`.

Faire. Recopiez ce début de classe **tel quel**, puis écrivez les méthodes du tableau.

```
@Service
public class EmpruntService {


    /** Durée maximale d'un emprunt, en jours (RM5). */
    public static final int DUREE_MAX_JOURS = 14;


    private final EmpruntRepository empruntRepository;
    private final LivreRepository livreRepository;


    public EmpruntService(EmpruntRepository empruntRepository, LivreRepository livreRepository) {
        this.empruntRepository = empruntRepository;
        this.livreRepository = livreRepository;
    }


    // Écrivez ici les méthodes du tableau
}
```

Signature	Ce que fait la méthode
`Page<Emprunt> recupererTous(Pageable pageable)`	Renvoie une page d'emprunts.
`Optional<Emprunt> recupererParId(Long id)`	Renvoie l'emprunt, ou un `Optional` vide.
`boolean livreExiste(Long livreId)`	Renvoie `true` si un livre porte cet id.
`boolean livreDisponible(Long livreId)`	**RM4** : renvoie `true` si le livre existe **et** que son stock est strictement supérieur à `0` ; sinon `false`.
`Emprunt creer(Emprunt emprunt)`	**RM5**, dans cet ordre : `id` = `null` ; `dateEmprunt` = aujourd'hui ; `dateRetourPrevue` = aujourd'hui + `DUREE_MAX_JOURS` ; `dateRetourEffective` = `null` ; `rendu` = `false` ; stock du livre − 1 et sauvegarde du livre ; sauvegarde de l'emprunt. Renvoie l'emprunt sauvegardé.
`Emprunt retourner(Emprunt emprunt)`	**RM6** : `rendu` = `true` ; `dateRetourEffective` = aujourd'hui ; si le livre existe encore, stock + 1 et sauvegarde du livre ; sauvegarde de l'emprunt. Renvoie l'emprunt sauvegardé.
`Page<Emprunt> recupererEnCours(Pageable pageable)`	Renvoie les emprunts non rendus (`findByRenduFalse`).
`Page<Emprunt> recupererEnRetard(Pageable pageable)`	**RM7** : renvoie les emprunts non rendus dont la date de retour prévue est avant aujourd'hui (`findByRenduFalseAndDateRetourPrevueBefore(LocalDate.now(), pageable)`).

> Pour modifier le stock du livre dans `creer` et `retourner` : récupérez-le avec `livreRepository.findById(emprunt.getLivreId())`, testez `isPresent()`, modifiez le stock avec `setStock(...)` puis appelez `livreRepository.save(livre)`.

Q29 — Le contrôleur `EmpruntController` : endpoints 15 à 20

Fichier. `controller/EmpruntController.java`.

Faire. Recopiez ce début de classe **tel quel**, puis écrivez les méthodes Java des endpoints **15 à 20** en suivant le plan en trois temps de la Q20.

```
@RestController
@RequestMapping("/api/v1/emprunts")
public class EmpruntController {

    private final EmpruntService empruntService;

    public EmpruntController(EmpruntService empruntService) {
        this.empruntService = empruntService;
    }

    // Écrivez ici les méthodes des endpoints 15 à 20
}
```

Règles.

- Listes : `@PageableDefault(size = 10, sort = "dateEmprunt") Pageable pageable`.
- Endpoint 17 : vérifiez d'abord `livreExiste` (sinon `400`), puis `livreDisponible` (sinon `409`).
- Endpoint 18 : l'annotation est `@PatchMapping("/{id}/retour")` ; il n'y a pas de corps. Après avoir récupéré l'emprunt, testez `emprunt.getRendu()` : s'il vaut `true`, renvoyez `409`.

Vérifier. Redémarrez l'application, puis exécutez dans l'ordre **toutes** les requêtes **R01 à R46**. Tous les codes doivent être conformes.

> **Fin du sprint 2.** Démonstration : les 46 requêtes de la recette sont conformes.

Partie G — Documenter l'API avec Swagger (Sprint 3, 30 min)

Q30 — Les informations générales de la documentation (Partie 4 §2.4)

Fichier. `config/OpenApiConfig.java`.

Faire. Copiez ce fichier **sans le modifier** :

```
package com.formation.bibliotheque.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Contact;
import io.swagger.v3.oas.models.info.Info;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI customOpenAPI() {
        Contact contact = new Contact();
        contact.setName("Équipe Formation");
        contact.setEmail("formation@exemple.fr");

        Info info = new Info();
        info.setTitle("API Bibliothèque");
        info.setVersion("1.0.0");
        info.setDescription("Projet fil rouge — gestion des auteurs, des livres et des emprunts");
        info.setContact(contact);

        OpenAPI openApi = new OpenAPI();
        openApi.setInfo(info);
        return openApi;
    }
}
```

Vérifier. `http://localhost:8080/swagger-ui.html` affiche le titre « API Bibliothèque » et la version 1.0.0.

Q31 — Annoter les trois contrôleurs (Partie 4 §2.4)

Fichiers. `AuteurController.java`, `LivreController.java`, `EmpruntController.java`.

Faire.

1. Ajoutez au-dessus de chaque classe contrôleur une annotation `@Tag` :

Contrôleur	Annotation
`AuteurController`	`@Tag(name = "Auteurs", description = "Gestion des auteurs")`
`LivreController`	`@Tag(name = "Livres", description = "Catalogue des livres")`
`EmpruntController`	`@Tag(name = "Emprunts", description = "Emprunts et retours des livres")`

2. Ajoutez au-dessus de **chacune des 20 méthodes** :
   - `@Operation(summary = "...")` : une phrase courte, de votre choix, qui décrit l'endpoint ;

- `@ApiResponses({...})` qui contient **une** `@ApiResponse` pour **chaque code différent** des colonnes « Succès » et « Erreurs » du §4.2, avec une description courte de votre choix.

Modèle pour l'endpoint 5 (codes `204`, `404` et `409`) :

```
@Operation(summary = "Supprimer un auteur (impossible s'il a encore des livres)")
@ApiResponses({
        @ApiResponse(responseCode = "204", description = "Auteur supprimé"),
        @ApiResponse(responseCode = "404", description = "Auteur inexistant"),
        @ApiResponse(responseCode = "409", description = "L'auteur a encore des livres")
})
@DeleteMapping("/{id}")
public ResponseEntity<Void> supprimer(@PathVariable Long id) {
    // ... (inchangé)
}
```

> Imports : `io.swagger.v3.oas.annotations.Operation`, `io.swagger.v3.oas.annotations.responses.ApiResponse`, `io.swagger.v3.oas.annotations.responses.ApiResponses`, `io.swagger.v3.oas.annotations.tags.Tag`.
>
> Un code qui apparaît deux fois dans la colonne « Erreurs » (par exemple `400` validation et `400` auteur inexistant, endpoint 9) ne donne **qu'une seule** `@ApiResponse` : décrivez les deux cas dans sa description.

Vérifier. Dans Swagger UI : trois groupes (Auteurs, Livres, Emprunts), 20 endpoints au total, et pour chaque endpoint la liste des codes du §4.2. Avec le bouton **Try it out**, exécutez `GET /api/v1/auteurs/{id}` avec `id = 1` : la réponse est `200`.

Partie H — La recette avec Postman (Sprint 3, 20 min)

Q32 — La collection de recette (Partie 1 §3.2 à §3.4)

Fichier à rendre, au choix. Soit la collection exportée `postman/Bibliotheque-API.postman_collection.json` (choix 1), soit le fichier `RECETTE.pdf` à la racine du projet (choix 2).

Faire. Vous avez enregistré les requêtes au fil des questions Q20, Q24, Q25 et Q29 ; complétez et organisez votre collection pour qu'elle respecte **exactement** ces règles :

1. la collection s'appelle `Bibliotheque-API` ;
2. elle définit une variable de collection `baseUrl` dont la valeur est `http://localhost:8080/api/v1`, et chaque URL commence par `{{baseUrl}}` (exemple : `{{baseUrl}}/auteurs/1`) ;
3. elle contient trois dossiers : `Auteurs` (R01 à R12), `Livres` (R13 à R28) et `Emprunts et fin du scénario` (R29 à R46) ;
4. elle contient les **46 requêtes** du §6, **dans l'ordre**, avec exactement la méthode, l'URL et le corps indiqués ;
5. chaque requête est nommée `<N°> — <MÉTHODE> <URL> → <code attendu>`, par exemple `R08 — DELETE /auteurs/1 → 409` ;
6. les requêtes avec un corps ont, dans l'onglet **Body**, le mode **raw** et le type **JSON**.

Vérifier. Redémarrez l'application, puis lancez toute la collection avec le **Collection Runner** de Postman (bouton **Run**). Pour chacune des 46 requêtes, le code obtenu doit être égal au code indiqué dans son nom.

Rendre (un seul des deux choix suffit).

- **Choix 1 — la collection.** Exportez-la (**Export** → *Collection v2.1*) dans `postman/Bibliotheque-API.postman_collection.json`.
- **Choix 2 — le PDF.** Créez `RECETTE.pdf` avec des captures d'écran de la page de résultats du Collection Runner. On doit y lire, pour **chacune des 46 requêtes**, son nom et le code HTTP obtenu : faites défiler la liste et prenez autant de captures que nécessaire. La première capture montre aussi le nom de la collection (`Bibliotheque-API`).

Partie I — Un client Java pour l'API (Sprint 3, 45 min)

Q33 — Le programme `ClientBibliotheque` (Partie 1 §3.5 et §3.6, Partie 2 §3.4)

Fichier. `client/ClientBibliotheque.java`, à la racine du projet, **hors** de `src/` : ce programme n'utilise pas Spring, il appelle votre API comme le ferait n'importe quel client. Il n'a pas de `package`.

Faire. Écrivez la classe `ClientBibliotheque` avec :

Élément	Déclaration exacte
Constante	`private static final String BASE_URL = "http://localhost:8080/api/v1";`
Client HTTP	`private static final HttpClient client = HttpClient.newHttpClient();`
Compteurs	`private static int nombreEtapes = 0;` et `private static int nombreConformes = 0;`
4 méthodes d'appel	`private static HttpResponse<String> get(String chemin)`, `post(String chemin, String corpsJson)`, `patch(String chemin)` et `delete(String chemin)`, toutes `throws Exception`
Affichage	`private static void afficher(String description, String methode, String chemin, HttpResponse<String> reponse, int codeAttendu)`
Programme	`public static void main(String[] args) throws Exception`

Règles des méthodes d'appel (modèle : `ClientRestSimple`, Partie 2 §3.4) :

- l'URL appelée est `BASE_URL + chemin` ;
- `get`, `post` et `patch` envoient l'en-tête `Accept: application/json` ; `post` envoie aussi `Content-Type: application/json` ;
- `HttpClient` n'a pas de méthode `.PATCH()` : dans `patch`, utilisez `.method("PATCH", HttpRequest.BodyPublishers.noBody())`.

Règles de `afficher`. La méthode ajoute 1 à `nombreEtapes`, ajoute 1 à `nombreConformes` si `reponse.statusCode()` est égal à `codeAttendu`, puis affiche **exactement** quatre lignes :

```
=== Étape <nombreEtapes> : <description> ===
<methode> <chemin> → <code reçu> (attendu : <codeAttendu>) <CONFORME ou NON CONFORME>
Réponse : <corps de la réponse>
<ligne vide>
```

Règles de `main`. Exécutez les 10 étapes ci-dessous, **dans l'ordre**, en appelant `afficher` après chaque requête. Écrivez les corps JSON avec des **Text Blocks** (`"""..."""`, Partie 1 §3.6). Terminez par la ligne `Bilan : <nombreConformes>/<nombreEtapes> étapes conformes`.

Étape	Description (texte à afficher)	Requête	Corps envoyé	Code attendu
1	Lister les 3 premiers livres disponibles	`GET /livres/disponibles?size=3`	—	`200`
2	Lire un livre inexistant	`GET /livres/99`	—	`404`
3	Créer l'auteur Jules Verne	`POST /auteurs`	`{"nom": "Verne", "prenom": "Jules", "nationalite": "Française"}`	`201`
4	Emprunter le livre 2	`POST /emprunts`	`{"livreId": 2, "emprunteur": "Alice Durand", "emailEmprunteur": "alice.durand@email.com"}`	`201`
5	Emprunter le livre 8 (stock à 0)	`POST /emprunts`	`{"livreId": 8, "emprunteur": "Alice Durand", "emailEmprunteur": "alice.durand@email.com"}`	`409`
6	Rendre l'emprunt 2	`PATCH /emprunts/2/retour`	—	`200`
7	Rendre l'emprunt 2 une deuxième fois	`PATCH /emprunts/2/retour`	—	`409`
8	Supprimer l'auteur 1 (il a des livres)	`DELETE /auteurs/1`	—	`409`
9	Créer un livre invalide (titre vide, prix négatif)	`POST /livres`	`{"titre": "", "isbn": "978-2-0000-0000-0", "prix": -5, "stock": 1, "auteurId": 1}`	`400`
10	Supprimer l'auteur 5 (aucun livre)	`DELETE /auteurs/5`	—	`204`

Vérifier. Redémarrez l'API (le scénario modifie les données), puis dans un terminal :

```
cd client
javac ClientBibliotheque.java
java ClientBibliotheque
```

Début de l'affichage attendu, puis dernière ligne :

```
=== Étape 1 : Lister les 3 premiers livres disponibles ===
GET /livres/disponibles?size=3 → 200 (attendu : 200) CONFORME
Réponse : {"content":[...],"page":{...}}

...

Bilan : 10/10 étapes conformes
```

Partie J — La soutenance (Jour 5, 15 h 30 – 16 h 45)

Chaque binôme dispose de **5 minutes de démonstration**, suivies de **2 minutes de questions**. Préparez la démonstration dans cet ordre :

Durée	Ce que vous montrez
1 min	Le client Java exécuté en direct sur une API redémarrée : `Bilan : 10/10 étapes conformes`.
1 min	Swagger UI : les trois groupes, puis un appel avec **Try it out**.
2 min	Le code de `EmpruntController.creer` et de `EmpruntService.creer` : expliquez l'ordre des vérifications et l'application des règles RM4 et RM5.
1 min	Votre réponse à une question de `CONCEPTION.pdf` choisie par le formateur.

Les 2 minutes de questions portent sur les Parties A à I (par exemple : « pourquoi `PATCH` ici ? », « que renvoie votre API si... ? », « à quoi sert ce DTO ? »).

6. La recette : 46 requêtes et leurs codes attendus

Ce tableau sert à **vérifier** votre API pendant tout le projet (Q20, Q24, Q25, Q29), à construire votre collection Postman (Q32) et à **évaluer** le projet.

Règles d'exécution.

- **Redémarrez l'API** avant chaque exécution complète : la base H2 est recréée et les identifiants repartent des valeurs du §3.3. Sans redémarrage, les codes ne sont plus les mêmes (un auteur déjà supprimé renvoie `404` au lieu de `204`...).
- Exécutez les requêtes **dans l'ordre** : certaines utilisent une ressource créée par une requête précédente (le livre R17 a pour auteur l'auteur 6, créé par R04).
- Les URLs sont écrites sans le préfixe : `/auteurs` signifie `http://localhost:8080/api/v1/auteurs` (`{{baseUrl}}/auteurs` dans Postman).
- Une requête est **conforme** si le code obtenu est égal au code attendu. La dernière colonne indique ce que vous pouvez contrôler en plus dans le corps de la réponse.

Dossier « Auteurs » (R01 à R12)

N°	Requête	Corps envoyé	Code attendu	Ce qui est vérifié
R01	`GET /auteurs`	—	**200**	5 auteurs ; le premier est « Asimov » (tri par nom)
R02	`GET /auteurs/1`	—	**200**	Victor Hugo
R03	`GET /auteurs/99`	—	**404**	Auteur inexistant
R04	`POST /auteurs`	`{"nom": "Vernes", "prenom": "Jules", "nationalite": "Française"}`	**201**	Auteur créé avec l'id 6 (faute de frappe volontaire : « Vernes »)
R05	`POST /auteurs`	`{"nom": "", "prenom": "Jules"}`	**400**	Nom vide : refusé par la validation
R06	`PUT /auteurs/6`	`{"nom": "Verne", "prenom": "Jules", "nationalite": "Française"}`	**200**	Nom corrigé en « Verne »
R07	`PUT /auteurs/99`	`{"nom": "Verne", "prenom": "Jules", "nationalite": "Française"}`	**404**	Auteur inexistant
R08	`DELETE /auteurs/1`	—	**409**	Victor Hugo a encore des livres

N°	Requête	Corps envoyé	Code attendu	Ce qui est vérifié
R09	`DELETE /auteurs/5`	—	**204**	Isaac Asimov n'a aucun livre : supprimé
R10	`DELETE /auteurs/5`	—	**404**	Déjà supprimé
R11	`GET /auteurs/1/livres`	—	**200**	Les 2 livres de Victor Hugo
R12	`GET /auteurs/99/livres`	—	**404**	Auteur inexistant

Dossier « Livres » (R13 à R28)

N°	Requête	Corps envoyé	Code attendu	Ce qui est vérifié
R13	`GET /livres?page=0&size=3&sort=prix,desc`	—	**200**	3 livres, le plus cher en premier (« Le Seigneur des Anneaux »)
R14	`GET /livres/1`	—	**200**	Les Misérables
R15	`GET /livres/99`	—	**404**	Livre inexistant
R16	`GET /livres/abc`	—	**400**	Identifiant non numérique (erreur automatique de Spring)
R17	`POST /livres`	`{"titre": "Vingt Mille Lieues sous les mers", "isbn": "978-2-253-00631-2", "prix": 7.9, "anneePublication": 1870, "stock": 2, "auteurId": 6}`	**201**	Livre créé avec l'id 9 (auteur : Jules Verne, id 6)
R18	`POST /livres`	`{"titre": "Vingt Mille Lieues sous les mers", "isbn": "978-2-253-00631-2", "prix": 7.9, "anneePublication": 1870, "stock": 2, "auteurId": 6}`	**409**	Même ISBN que le livre 9
R19	`POST /livres`	`{"titre": "Livre fantôme", "isbn": "978-2-0000-0000-1", "prix": 10.0, "anneePublication": 2020, "stock": 1, "auteurId": 99}`	**400**	L'auteur 99 n'existe pas
R20	`POST /livres`	`{"titre": "", "isbn": "978-2-0000-0000-2", "prix": -3, "stock": 1, "auteurId": 1}`	**400**	Titre vide et prix négatif : refusé par la validation

N°	Requête	Corps envoyé	Code attendu	Ce qui est vérifié
R21	`PUT /livres/9`	`{"titre": "Vingt Mille Lieues sous les mers", "isbn": "978-2-253-00631-2", "prix": 8.5, "anneePublication": 1870, "stock": 2, "auteurId": 6}`	**200**	Prix modifié (8.5) ; garder son propre ISBN est autorisé
R22	`PUT /livres/9`	`{"titre": "Vingt Mille Lieues sous les mers", "isbn": "978-2-07-040850-4", "prix": 8.5, "anneePublication": 1870, "stock": 2, "auteurId": 6}`	**409**	ISBN déjà porté par le livre 1
R23	`PUT /livres/99`	`{"titre": "Livre inconnu", "isbn": "978-2-0000-0000-3", "prix": 10.0, "anneePublication": 2020, "stock": 1, "auteurId": 1}`	**404**	Livre inexistant
R24	`GET /livres/recherche?titre=harry`	—	**200**	Les 2 livres « Harry Potter »
R25	`GET /livres/recherche`	—	**400**	Paramètre titre absent (erreur automatique de Spring)
R26	`GET /livres/disponibles`	—	**200**	8 livres disponibles (tous sauf « Le Hobbit »)
R27	`PATCH /livres/8/stock?quantite=0`	—	**400**	Quantité nulle refusée
R28	`PATCH /livres/99/stock?quantite=2`	—	**404**	Livre inexistant

Dossier « Emprunts et fin du scénario » (R29 à R46)

N°	Requête	Corps envoyé	Code attendu	Ce qui est vérifié
R29	`POST /emprunts`	`{"livreId": 8, "emprunteur": "Alice Durand", "emailEmprunteur": "alice.durand@email.com"}`	**409**	« Le Hobbit » a un stock de 0
R30	`POST /emprunts`	`{"livreId": 99, "emprunteur": "Alice Durand", "emailEmprunteur": "alice.durand@email.com"}`	**400**	Le livre 99 n'existe pas
R31	`POST /emprunts`	`{"livreId": 2, "emprunteur": "Alice Durand", "emailEmprunteur": "alice.durand"}`	**400**	Email invalide : refusé par la validation
R32	`POST /emprunts`	`{"livreId": 2, "emprunteur": "Alice Durand", "emailEmprunteur": "alice.durand@email.com"}`	**201**	Emprunt créé avec l'id 4 ; rendu = false ; enRetard = false ; pas d'email dans la réponse

N°	Requête	Corps envoyé	Code attendu	Ce qui est vérifié
R33	`GET /livres/2`	—	**200**	Le stock du livre 2 est passé de 3 à 2
R34	`GET /emprunts/4`	—	**200**	L'emprunt 4, sans email
R35	`GET /emprunts/99`	—	**404**	Emprunt inexistant
R36	`GET /emprunts/en-cours`	—	**200**	3 emprunts non rendus (id 2, 3 et 4)
R37	`GET /emprunts/en-retard`	—	**200**	2 emprunts en retard (id 2 et 3)
R38	`PATCH /emprunts/2/retour`	—	**200**	Emprunt 2 rendu : rendu = true, dateRetourEffective = aujourd'hui
R39	`PATCH /emprunts/2/retour`	—	**409**	Déjà rendu
R40	`PATCH /emprunts/99/retour`	—	**404**	Emprunt inexistant
R41	`PATCH /livres/8/stock?quantite=2`	—	**200**	« Le Hobbit » réapprovisionné : stock = 2
R42	`POST /emprunts`	`{"livreId": 8, "emprunteur": "Paul Morel", "emailEmprunteur": "paul.morel@email.com"}`	**201**	« Le Hobbit » peut maintenant être emprunté
R43	`GET /emprunts`	—	**200**	5 emprunts au total
R44	`DELETE /livres/9`	—	**204**	Livre 9 supprimé
R45	`DELETE /livres/9`	—	**404**	Déjà supprimé
R46	`GET /inexistant`	—	**404**	URL inconnue (erreur automatique de Spring)

7. Grille d'évaluation

Critère	Points	Ce qui est vérifié	Questions
Dossier de conception	/4	Réponses justes et justifiées aux questions Q1 à Q9	Q1 à Q9
Initialisation et configuration	/2	Projet généré avec les bons paramètres ; l'application démarre ; les données initiales sont chargées	Q10, Q11, Q16
Entités et repositories	/4	Annotations JPA conformes au §3.2 ; ISBN unique ; méthodes des repositories exactes	Q12 à Q15
DTOs et mappers	/4	Aucune entité renvoyée au client ; email absent des réponses (RM8) ; ordre des champs respecté ; mappers avec boucles `for`	Q17, Q18, Q21, Q22, Q26, Q27
Validation des entrées	/3	Contraintes et messages du §4.3 ; `@Valid` sur chaque corps reçu ; requêtes R05, R20 et R31 conformes	Q17, Q21, Q26
Règles métier dans les services	/6	RM2 à RM7 appliquées dans les services : stock, dates à J+14, retour unique, retard, suppression d'un auteur	Q19, Q23, Q28
Contrôleurs et codes HTTP	/6	20 endpoints ; plan en trois temps ; ordre des vérifications ; **nombre de requêtes conformes sur les 46 de la recette**	Q20, Q24, Q25, Q29
Pagination	/3	`PagedModel` sur les 8 listes ; tailles et tris par défaut du §4.1 ; `page`, `size` et `sort` fonctionnent	Q18, Q20, Q24, Q29
Documentation Swagger	/2	Titre et version ; 3 groupes ; `@Operation` et `@ApiResponses` complets sur les 20 endpoints	Q30, Q31
Recette Postman	/1	Collection exportée conforme aux règles de la Q32, **ou** `RECETTE.pdf` montrant les 46 résultats du Collection Runner ; codes conformes	Q32
Client Java	/2	`HttpClient` utilisé correctement (4 méthodes, en-têtes, `PATCH`) ; affichage conforme ; bilan 10/10	Q33
Qualité du code	/1	Code lisible (nommage clair, indentation) ; couches respectées (pas d'appel à un repository depuis un contrôleur)	Tout le projet
Soutenance	/2	Démonstration fluide ; explications claires ; réponses justes aux questions	Partie J
Total	**/40**		

> Pour obtenir une note sur 20, divisez le total par 2.

8. Checklist avant de rendre

- ☐ `CONCEPTION.pdf` contient une réponse à chacune des questions Q1 à Q9.
- ☐ L'application démarre sans erreur et charge 5 auteurs, 8 livres et 3 emprunts.
- ☐ Aucune méthode de contrôleur ne renvoie une entité (`Auteur`, `Livre`, `Emprunt`) : seulement des DTOs de réponse.
- ☐ `emailEmprunteur` n'apparaît dans aucune réponse de l'API.

- ☐ Les 46 requêtes de la recette sont conformes, exécutées dans l'ordre sur une API fraîchement redémarrée.
- ☐ Swagger UI affiche les 3 groupes et les 20 endpoints, chacun avec ses codes de réponse.
- ☐ Le client Java affiche `Bilan : 10/10 étapes conformes` sur une API fraîchement redémarrée.
- ☐ L'archive porte le bon nom, contient `CONCEPTION.pdf`, `client/`, `src/` et, pour la recette, `postman/` **ou** `RECETTE.pdf`, mais **pas** le dossier `target/`.